A AEGIS LAB / ENGINEERING PORTFOLIO

BUILD SYSTEMS, NOT JUST DEMOS

AEGIS-SIM

AI-Powered Emergency Response Simulation
& Decision-Support Platform

একটি বিশ্বমানের **C# / .NET Software Engineering Portfolio Project** তৈরির সম্পূর্ণ বাংলা গাইড

Architecture · Simulation Engine · OOP · Database · Real-time UI · Responsible AI · Explainability · Replay · Testing · Deployment · Interview Demo

PRODUCT REQUIREMENTS | SYSTEM DESIGN | BUILD ROADMAP | AI CODING PROMPTS

প্রস্তাবিত প্রকল্প
নিরাপদ, সম্পূর্ণ কল্পিত দুর্যোগ-সাড়া প্রশিক্ষণ সিমুলেশন

সংস্করণ
v1.0 · ২২ অক্টোবর ২০২৬

READ THIS FIRST / 00

শুরু করার আগে: একটি গুরুত্বপূর্ণ সিদ্ধান্ত

পরিধি ও দাবি সম্পর্কে সতর্কতা

এই PDF একটি

পোর্টফোলিও ও শেখার ব্লুপ্রিন্ট

, কোনো অফিসিয়াল Army War Game Centre সিস্টেমের specification নয়। বিজ্ঞপ্তিতে C#/.NET-কে অগ্রাধিকার, OOP, Database ও Simulation/Software-এর সঙ্গে সম্পর্কিত দক্ষতার কথা এসেছে; প্রকৃত অভ্যন্তরীণ কাজের বিস্তারিত প্রকাশিত হয়নি। এই প্রকল্পে কেবল

সিনথেটিক দুর্যোগ-সাড়া, উদ্ধার ও সিভিল লজিস্টিকস

দেখানো হবে—কোনো যুদ্ধকৌশল, অস্ত্র, লক্ষ্যবস্তু নির্বাচন বা বাস্তব সামরিক অপারেশন নয়।

মূল লক্ষ্য: ৩ মিনিটের ডেমোতে এমন একটি চালু সিস্টেম দেখানো, যেখানে Simulation Start → Scenario Change → Real-time Outcome → Data Save → Replay → Report ধারাটি ভেঙে যায় না। অতিরিক্ত AI ফিচার আসবে এই ভিত্তি শক্ত হওয়ার পরে।

01
Working System
UI + Engine + Database একসঙ্গে

02
Explainable Architecture
OOP, DI, Modular Design, Tests

03
Trustworthy AI
Baseline, Metrics, Uncertainty

এটি কী, আর কী নয়

এটি হবে	এটি হবে না
Offline-friendly browser-based synthetic training simulator	বাস্তব যুদ্ধ পরিকল্পনা বা সামরিক সিদ্ধান্ত নেওয়ার টুল
নিয়ন্ত্রিত, **reproducible scenario engine**	শুধু সুন্দর Dashboard বা LLM Chatbot
কোড, **Test, Demo Video, GitHub সহ portfolio artifact**	অপ্রমাণিত “state-of-the-art” AI দাবি
Recruitment-relevant programming evidence	নিয়োগ বা চাকরি পাওয়ার নিশ্চয়তা

কম সময় পেলে কোনগুলো করব?

Minimum Winning Demo: ১টি Scenario, ৮–১২টি synthetic Agent, Start/Pause/Resume/Reset, ২টি Incident, একটি what-if change, Database persistence, এক ক্লিকে Replay, ৫টি Metrics ও ১০টি Test। AI না থাকলেও এটি একটি শক্তিশালী Engineering Demo হতে পারে।

BUILD BLUEPRINT / 01

নিয়োগ বিজ্ঞপ্তির সঙ্গে Skill-Mapping

চাহিদা / প্রাসঙ্গিক বিষয়	AEGIS-SIM-এ প্রমাণ	প্রমাণ হিসেবে কী দেখাবে
OOP-এ পারদর্শিতা	Entity, SimulationEngine, Rule, Event, Service interface	Class diagram + unit test
C# / .NET অগ্রাধিকার	Core engine, ASP.NET Core API, Blazor UI	C# solution + live demonstration
Database ব্যবহারিক জ্ঞান	Scenarios, Runs, Events, Snapshots, Metrics	EF Core migrations + query + persisted replay
Software/Solution engineering	Clean separation, dependency injection, error handling	Architecture diagram + README
Simulation-ধর্মী কাজ	Deterministic ticks, scenario states, what-if analysis	Run A বনাম Run B comparison
বোনাস দক্ষতা	ML baseline, uncertainty, evidence-linked summary	Measured evaluation + explainability view

বিজ্ঞপ্তিতে

C#/.NET অগ্রাধিকার

বলা হয়েছে—অবশ্যই জানা বাধ্যতামূলক এমন সিদ্ধান্ত নেওয়া যাবে না। তবে এই পদের সঙ্গে শক্তিশালী মিল দেখাতে C#/.NET ব্যবহার করাই সবচেয়ে যুক্তিযুক্ত।

ইন্টারভিউ প্যানেলকে ৬টি Engineering প্রশ্নের উত্তর দিতে হবে

- Requirement থেকে working feature বানাতে পারি কি?
- Complex state, timing ও events ঠিকভাবে manage করতে পারি কি?
- OOP abstraction এবং testable code লিখতে পারি কি?
- DB, API ও UI reliably integrate করতে পারি কি?
- Bug, failure, restart ও replay সামলাতে পারি কি?
- AI output যাচাই ও সীমাবদ্ধতা ব্যাখ্যা করতে পারি কি?

BUILD BLUEPRINT / 02

Product Vision, Personas ও বাস্তব ব্যবহারপ্রবাহ

Problem statement: প্রশিক্ষককে বিভিন্ন কল্পিত জরুরি পরিস্থিতি তৈরি, চালানো, পরিবর্তন, তুলনা এবং বিশ্লেষণ করতে সহায়তা করা; নতুন শিক্ষার্থীদের নিয়ম-ভিত্তিক Simulation কীভাবে কাজ করে তা দৃশ্যমান করা।

Persona	মূল কাজ	অ্যাক্সেস
Instructor / Controller	Scenario তৈরি, run control, parameter edit, review	Write + Control
Observer / Evaluator	Live view, charts, timeline, post-run report	Read + Export
Developer / Maintainer	Test, logs, seed reproducibility, model evaluation	Admin (local/demo)

প্রথম ৩টি Synthetic Scenario

Scenario A · Flood Shelter Coordination

MVP

৩টি shelter, ৮টি relief vehicle, সীমিত supplies; রাস্তা বন্ধ হলে estimated service completion বদলায়।

Done when: run, pause, incident injection ও KPI update কাজ করে।

Scenario B · Earthquake Medical Logistics

POLISH

চিকিৎসা সরঞ্জাম বিতরণ, স্টক শেষ হওয়ার alarm, বিকল্প depot থেকে বিতরণের simulation।

Done when: দুটি run তুলনা করা যায়।

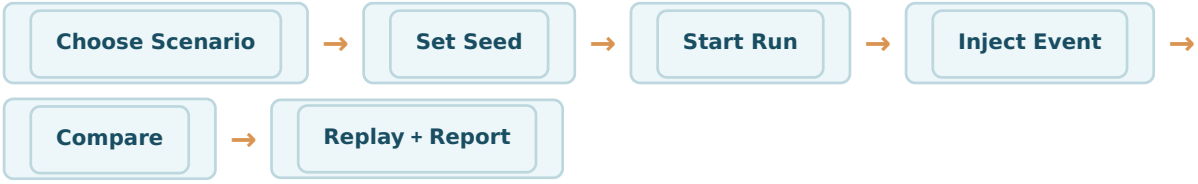
Scenario C · Cyclone Communication Outage

STRETCH

যোগাযোগ delay ও তথ্যের অনিশ্চয়তা simulation; AI prediction confidence কমে গেলে সতর্ক বার্তা।

Done when: uncertainty correctly displayed।

Core user journey



BUILD BLUEPRINT / 03

Feature Inventory: MVP → Portfolio → Advanced

F01 · Scenario Catalog + Validation — MVP

Create/load/edit a JSON scenario: map dimensions, resources, incident schedule, rules, seed. Reject invalid schema.

Done when: ৩টি seed scenario load হয়; invalid input 400/validation error।

F02 · Deterministic Simulation Engine — MVP

Fixed time step; no wall-clock dependency in domain logic; state transitions and ordered event processing.

Done when: same scenario + same seed → identical final state & event hash।

F03 · Interactive SVG Map — MVP

নিজস্ব synthetic grid map, shelter nodes, route lines, animated civilian resource icons; no internet required.

Done when: offline demo-তে map ও state sync থাকে।

F04 · Playback Controls — MVP

Start / Pause / Resume / Step / Reset / Speed (1×, 2×, 4×); only valid transitions.

Done when: start থেকে ৫০ ticks কোনো exception ছাড়াই।

F05 · Incident Injection — MVP

Flooded route, depot outage, vehicle unavailable; simulation event updates capacity/availability.

Done when: before/after metrics visibly change; event audit-এ লেখা হয়।

F06 · Event Log + Snapshot + Replay — MVP

Typed events persist; configurable snapshots; historic read-only replay can jump to tick.

Done when: save → restart app → same history replay।

F07 · KPI Dashboard — MVP

Completion %, average waiting time, active resources, deliveries completed, resource utilization.

Done when: verified calculations এবং refresh।

F08 · A/B Scenario Comparison — POLISH

Keep same seed and baseline; vary one independent parameter; side-by-side outcomes.

Done when: difference table, delta bar, downloadable report।

F09 · Multi-agent Civilian Logistics

POLISH

Agent abstractions: dispatch coordinator, supply monitor, shelter monitor; initial logic rule-based.

Done when: each action traceable to a rule/reason।

F10 · Predictive ML

POLISH

Predict a completion-time estimate using synthetic historical runs; compare against simple baseline.

Done when: holdout MAE and limitation shown।

F11 · Explainability + AI Assistant

POLISH

Feature contributions when supported; evidence-linked natural-language summary from recorded metrics.

Done when: AI never invents an event; can turn off LLM and still generate report।

F12 · Authentication and Roles

ADVANCED

Instructor/Observer roles, action authorization, run ownership, rate limit.

Done when: negative authorization tests pass।

F13 · Failure Recovery + Observability

ADVANCED

Structured logs, health endpoint, cancellation, checkpoint recovery, run metrics.

Done when: controlled restart test passes।

F14 · Export / Accessibility / Deployment

ADVANCED

PDF/CSV export, keyboard controls, contrast, deployment docs, GitHub CI.

Done when: fresh machine clone-and-run successful।

BUILD BLUEPRINT / 04

Non-Functional Requirements ও Acceptance Targets

Quality attribute	প্রথম লক্ষ্য / যাচাই
Determinism	Same seed, same config and engine version => same result hash (single-thread core)
Responsiveness	ল্যাপটপে ১০–২৫ entities, ১০ Hz state updates এ UI ব্যবহারযোগ্য; পরিমাপ করে লিখবে
Persistence	Run stop / browser refresh-এর পর event timeline অক্ষত
Correctness	Invalid transitions rejected; negative stock and impossible resource state prohibited
Offline mode	Synthetic SVG map এবং deterministic engine external tile/service ছাড়াই চলে
Testability	Core rules unit-tested; ১টি end-to-end demo pathway automation
Explainability	Metrics calculation reproducible; AI output references verified run facts
Security	User input validation, parameter limits, parameterized EF queries, no secrets in repo
Reproducibility	README setup, sample data, seed and versioned scenario file
Portability	Windows development + documented optional Docker deployment

মেট্রিকস নিয়ে সততা

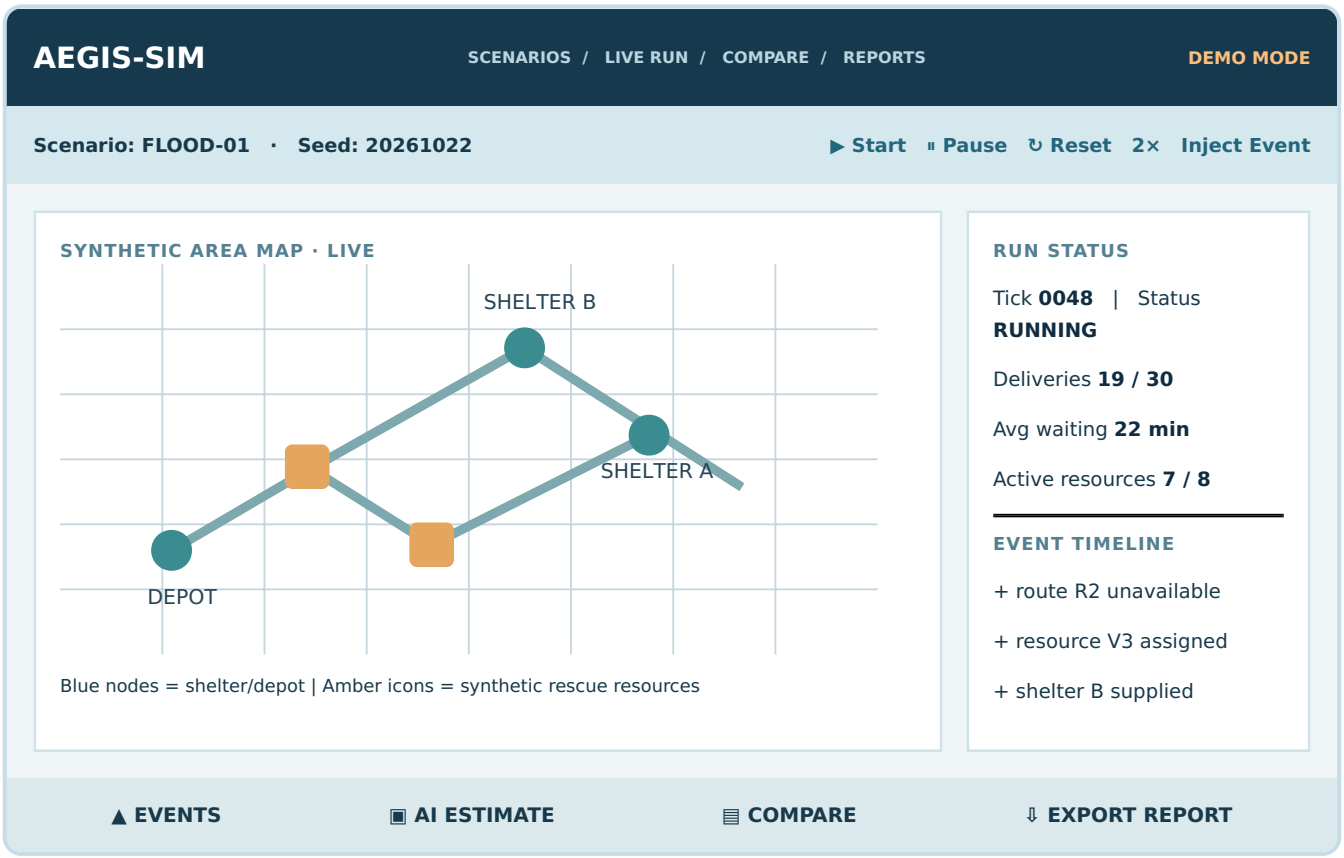
এগুলো

নিজের প্রজেক্টের target

, আগে থেকে অর্জিত benchmark নয়। Build শেষে বাস্তবে measurement নিয়ে README-তে লিখবে।

BUILD BLUEPRINT / 05

UI/UX Blueprint: এক স্ক্রিনে পুরো কাহিনি



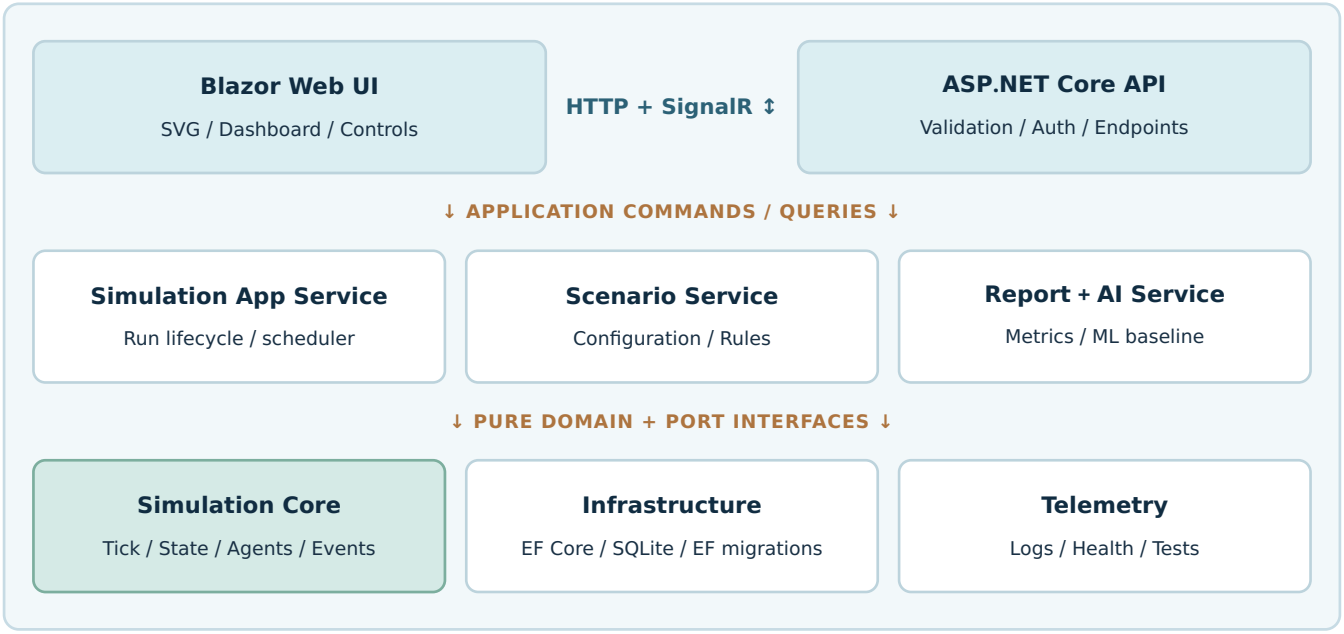
Page / Component breakdown

Page	Widgets	Priority
Overview	Recent runs, scenario library, 4 KPI tiles	MVP
Run Workspace	SVG Grid, Controls, Event feed, per-agent panel	MVP
Run History	Run list, status, seed, duration, replay	MVP
Comparison	baseline vs modified run, metric deltas, charts	POLISH
AI Inspector	input features, baseline, prediction, uncertainty	POLISH
Reports	narrative + evidence + export	POLISH
Admin	validation logs, configuration, roles	ADVANCED

UX rules: Show current simulation tick; disable impossible buttons; distinguish observed metric vs predicted metric; include “synthetic training data” watermark; show an error state with a user-friendly message and correlation ID.

BUILD BLUEPRINT / 06

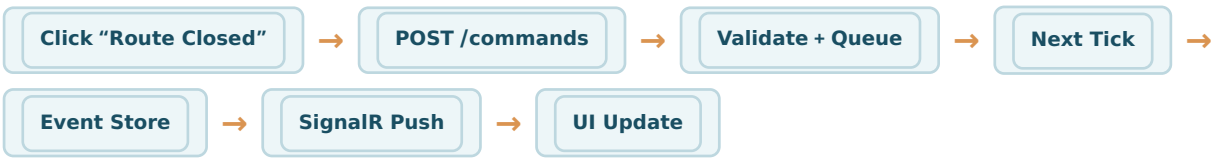
End-to-End System Architecture



কেন এই Design?

- **UI Engine নয়:** Simulate() কোনো Blazor component-এর মধ্যে লিখবে না; তাই command line-এও engine test করা যাবে।
- **Dependency rule:** Domain → কোনো DB, HTTP বা UI জানবে না। Infrastructure domain interfaces implement করবে।
- **Backend authoritative:** simulation state backend-এ; UI কেবল view/control।
- **Single writer:** একটি run-এর tick loop এক সময়ে একজন writer চালাবে; command queue serialized হবে।
- **Separately measurable AI:** AI module বন্ধ থাকলেও core system সম্পূর্ণ চালু।

একটি Event-এর Journey



BUILD BLUEPRINT / 07

Repository Structure ও Development Workflow

```text
AegisSim/
├── src/
│   ├── AegisSim.Domain/          # Entity, Value Object, Rule, Event
│   ├── AegisSim.Application/     # Run lifecycle, commands, DTOs
│   ├── AegisSim.Infrastructure/  # EF Core, persistence, AI adapters
│   ├── AegisSim.Api/             # REST, SignalR, DI, hosting
│   └── AegisSim.Web/             # Blazor pages + SVG components
├── tests/
│   ├── AegisSim.Domain.Tests/
│   └── AegisSim.Api.Tests/
├── data/scenarios/              # synthetic versioned JSON
├── docs/                        # ADRs, architecture, demo, metrics
├── scripts/                     # setup and seed
├── .github/workflows/           # build/test CI
├── README.md
├── SECURITY.md
└── AegisSim.sln
```

Branch ও commit routine

- প্রথমে `main` stable রাখবে; ছোট feature branch: `feat/simulation-tick`।
- প্রতি feature-এ issue → acceptance criteria → code → test → commit → screenshot।
- কমিট বার্তা: `feat(engine): add deterministic route closure events`।
- AI generated code নিজে review করে তারপর commit; failure screenshots ও fix rationale লিখে রাখবে।

BUILD BLUEPRINT / 08

OOP Domain Model: Class, Responsibility, Invariant

Type	Responsibility	Invariant / Rule
ScenarioConfig	initial map, seed, supply, agent list, schedule	dimensions positive; IDs unique
SimulationRun	lifecycle, tick count, version, metadata	no transition from Completed to Running
SimulationState	current inventory, statuses, progress	supply ≥ 0; tick monotonic
ResourceAgent	capacity, status, assigned request	one active assignment per resource
ShelterDemand	required quantity, delivered, priority bucket	delivered ≤ required (unless overfill explicitly permitted)
ISimulationRule	Advance state for one logical tick	pure deterministic transition
IScenarioEvent	typed change e.g. RouteClosed	valid payload; unique ID
ISnapshotStore	write/read state checkpoints	checkpoints versioned
IMetricCalculator	compute observed KPI from state/events	same inputs => same output
IPredictionService	predict ETA from feature vector	returns version, confidence/ interval, provenance

SOLID কোথায় কাজে লাগবে?

- **S:** ScenarioLoader config পড়ে; TickEngine শুধুই state advance করে।
- **O:** new event handler যোগ করলেও core scheduler বদলাতে হয় না।
- **L:** IMetricCalculator implementation বদলালেও same contract থাকবে।
- **I:** read-only viewer interface আলাদা, controller interface আলাদা।
- **D:** app service `IRunRepository` interface-এ নির্ভর করবে, EF context-এ নয়।

State lifecycle



Allow `Running → Failed` on nonrecoverable error; allow `Paused → Cancelled`. Reset creates a **new run** rather than silently mutating completed history.

BUILD BLUEPRINT / 09

Deterministic Simulation: ভিতরের Engine কীভাবে কাজ করবে

Fixed-tick principle

Wall clock দিয়ে rule calculation করবে না। প্রতিটি tick-এ virtual time একটি নির্ধারিত পরিমাণ অগ্রসর হবে। UI speed 1×/2×/4× কেবল real-time scheduling বদলাবে, simulation outcome নয়।

Pseudocode

```
Input: current State S[t], queued Commands C[t], Seeded RNG
1. Validate and order commands deterministically
2. Apply scheduled scenario events
3. Update resource availability and demand
4. Run civilian dispatch rule (if eligible)
5. Advance task progress for one virtual tick
6. Generate typed domain events
7. Check invariants and compute metrics
8. Persist events; snapshot every N ticks
9. Publish read-only DTO to viewers
Output: S[t+1], ordered events, metrics
```

Compilable C# micro-engine example (training only)

C# / immutable state transition

```
public record SimState(int Tick, int DemandLeft, int Delivered);
public record SimEvent(int Tick, string Type, int Amount);

public static class MiniEngine
{
    public static (SimState State, SimEvent? Event) Step(
        SimState s, int availableCapacity)
    {
        if (availableCapacity < 0) throw new ArgumentOutOfRangeException();
        int moved = Math.Min(s.DemandLeft, availableCapacity);
        var next = s with {
            Tick = s.Tick + 1,
            DemandLeft = s.DemandLeft - moved,
            Delivered = s.Delivered + moved
        };
        SimEvent? evt = moved == 0 ? null :
            new SimEvent(next.Tick, "SupplyDelivered", moved);
        return (next, evt);
    }
}
```

এই উদাহরণটি intentionally ছোট: বাস্তব project-এ entity lists, commands, scenario policies, event ordering, concurrency protection এবং persistence আলাদা করতে হবে।

৫টি অপরিহার্য Unit Test

- Same inputs and seed → identical next and final states (verify event hash).
- Negative capacity input throws; demand never negative; delivery ≤ capacity.
- One tick increments Tick exactly once; transitions remain valid.

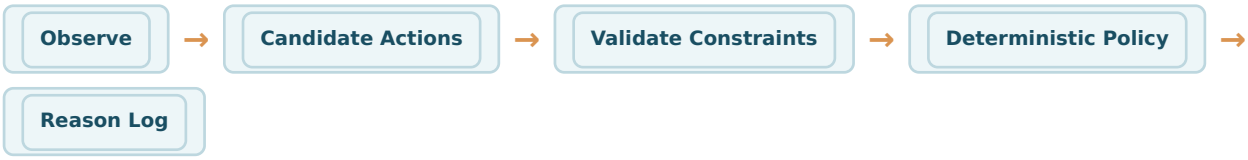
BUILD BLUEPRINT / 10

Agent System: AI শব্দের আড়ালে Engineering

এখানে Agent মানে state দেখে নিয়ম/মডেল অনুযায়ী action proposal করা একটি software component। **Rule-based agents-কে "trained autonomous AI" বলা যাবে না।** আগে explainable rules; পরে প্রয়োজন হলে ML-powered advisory layer।

Agent	Observe	Suggest	Audit reason
ResourceAgent	capacity, availability, queue	accept/hold assigned supply task	available resource + sufficient capacity
ShelterMonitorAgent	unmet demand, service history	request more supply	demand still positive
SupplyMonitorAgent	depot stock and replenishment schedule	warn low stock	stock below configurable threshold
CoordinatorAgent	pending tasks, current utilization	assign next available civilian resource	rules-ranked request and availability

Decision Pipeline



Rule-baseline first

MVP

Explainable FIFO or earliest-request-first resource assignment. Keeps behavioral logic easily testable.

Done when: decision reasons saved.

Advanced (optional): bounded learned ETA

ADVANCED

Model only estimates task completion; final resource movement still governed by verified simulator rules.

Done when: model disabled toggle; no undocumented action change.

BUILD BLUEPRINT / 11

Data Architecture: Tables, Keys, Relationships

Table	Selected fields	Notes
Scenarios	id, name, version, schemaJson, createdAt	version immutable after publication
Runs	id, scenarioId, seed, engineVersion, status, currentTick	one run belongs to one scenario version
RunCommands	id, runId, requestedTick, type, payloadJson, actorId	validated requests, idempotency key
RunEvents	id, runId, tick, seq, type, payloadJson, hash	append-only per run; seq unique
Snapshots	id, runId, tick, stateJson, checksum	every configurable 25 ticks
RunMetrics	id, runId, tick, name, value, unit	computed from observed state
PredictionRecords	id, runId, tick, modelVersion, pred, lower, upper	prediction provenance / nullable intervals
ModelVersions	id, trainingDataHash, algorithm, testMae, createdAt	no invented accuracy
Users (advanced)	id, role, credential reference	auth only when polished

Relationships / constraints

- `Scenario 1 : N Runs`, `Run 1 : N Events`, `Run 1 : N Snapshots`.
- Unique index: `RunEvents(runId, tick, seq)`; `Runs(scenarioId)` indexed.
- Snapshot JSON carries `schemaVersion`; replay rejects incompatible version instead of corrupting state.
- SQLite first: zero setup; SQL Server optional later. SQLite has provider-specific migration limitations, so keep repository abstractions clean.

Replay algorithm

Pseudocode: replay

```
targetTick = 190
snapshot = latest snapshot WHERE tick <= targetTick
state = deserialize(snapshot.stateJson)
for evt in events ORDER BY (tick, seq) between snapshot.tick and targetTick:
    state = ApplyEvent(state, evt)
assert Hash(state) == expectedHash(targetTick)
return state
```

> Replay architecture-এ event যথেষ্ট information বহন করতে হবে। শুধু “something changed” টেক্সট লগ রাখলে নির্ভরযোগ্য state reconstruction হবে না।

BUILD BLUEPRINT / 12

API Contract ও Real-time Messaging

Method	Endpoint	Purpose / return
GET	/api/scenarios	Scenario summaries
POST	/api/scenarios/validate	Validate scenario JSON; field-level errors
POST	/api/runs	Create run with scenarioId + seed; returns runId
GET	/api/runs/{id}	Run summary and current state
POST	/api/runs/{id}/commands	Start/Pause/Resume/Step/Inject (validated)
GET	/api/runs/{id}/events?after=…	Ordered event log, pagination
GET	/api/runs/{id}/snapshot?tick=…	Historic read-only state
GET	/api/runs/{id}/metrics	Observed KPI series
POST	/api/compare	Compare two completed runs
GET	/api/runs/{id}/report	Summary, provenance and export links
GET	/health	Health status

Example command payload

JSON / illustrative contract

```
POST /api/runs/3d1.../commands
{
  "commandId": "cmd-0042",
  "type": "InjectRouteClosure",
  "expectedRunVersion": 7,
  "payload": { "routeId": "R2", "durationTicks": 15 }
}

202 Accepted
{ "accepted": true, "commandId": "cmd-0042" }
```

SignalR Hub: /hubs/runs

- Groups named per run: `run:{runId}`; authorize group subscription when roles exist.
- Server → client: `StateUpdated`, `EventCommitted`, `RunStatusChanged`, `MetricsUpdated`.
- First HTTP snapshot, then subscribe; on reconnect fetch missed events with cursor to close gaps.
- State snapshots max ~5–10/second for MVP; tune only after profiling.
- Do not trust arbitrary command payloads sent from browsers; process validated commands in application service.

BUILD BLUEPRINT / 13

AI/ML Design: সত্যিকারের মূল্য কীভাবে তৈরি হবে

Task: completion-time prediction

Target variable = **remaining virtual minutes to complete pending deliveries**. Input candidate features: unmet demand, available resources, depot stock, blocked-route count, recent service rate, queued tasks. All are synthetic and available only up to the prediction tick.

Stage	Approach / safeguard
Dataset	Create many deterministic scenario variants; save configuration and generation seeds
Baseline #1	Constant global median remaining duration
Baseline #2	Simple throughput heuristic: demand / recent delivery rate (guard zero)
ML candidate	ML.NET regression (FastTree/SDCA or simple linear), evaluate before selection
Split	Hold out whole scenario families/config groups, not random ticks from same run
Metrics	MAE = average absolute error; secondary RMSE and error by scenario
Uncertainty	If supported/calibrated, interval from held-out residual quantiles; otherwise show no interval
Monitoring	Compare live predictions against observed outcomes; flag out-of-range inputs
UI wording	Predicted duration ≠ actual duration; show model version and measured MAE

Leakage traps

- **Forbidden:** future event, final completion count, or actual finish time inside features at tick t.
- Train/validation/test must not contain ticks from the same run in different splits.
- Do not use an LLM to invent quantitative performance; tests must produce numbers.
- All gains need comparison against a baseline on held-out scenarios; no guarantee of superiority.

LLM reporting: grounded, optional

LLM-এর কাজ কেবল logged facts দিয়ে readable summary লেখা, সিদ্ধান্ত বা simulation-state update নয়। Prompt context-এ runId, source event IDs, observed KPI JSON, prediction provenance থাকবে; output structured JSON validator দিয়ে যাচাই করে UI-তে দেখাবে। Internet বা API key না থাকলে deterministic template fallback।

> **কী দিয়ে আলাদা হবে**
>
> AI “ম্যাজিক” নয়। Benchmarked ML baseline + reproducible scenario + trustworthy explanation—এই তিনটি জিনিস recruitment panel-এর কাছে বেশি credible।

BUILD BLUEPRINT / 14

Analytics, Comparison ও After-Action Report

Metric	Operational definition (training sim)
Task completion rate	completed assigned tasks / total valid tasks × 100
Average waiting time	mean(first_service_tick – request_created_tick) × tick_minutes
Resource utilization	busy resource-ticks / available resource-ticks × 100
Total duration	(last_tick – first_tick) × tick_minutes
Unmet demand	Σ max(required – delivered, 0)
Prediction MAE	Σ |predicted remaining time – observed remaining time| / N

Fair A/B comparison

- Keep **same scenario version and seed** for paired comparison; change only one configured parameter.
- Show direction and magnitude: `Δ wait = wait_B - wait_A`, units required.
- Highlight trade-offs, not only improvements (e.g. lower waiting but higher utilization).
- Compare ৫–২০ seeds as an advanced option; mean and distribution beats a single lucky seed.

Report structure

01 Scenario Identity

02 Configuration + Seed

03 Observed KPIs

04 Key Event Timeline

05 A/B Delta

06 Prediction + Error

07 Limitations

08 Evidence Links

BUILD BLUEPRINT / 15

Security, Reliability, Responsible Demo

Must-have protection

- Input limit: max map size, max entities, max incident frequency, max steps per run.
- Validate all scenario configuration server-side; reject malformed IDs, NaN and negative capacities.
- Use ASP.NET authorization at both API and SignalR hub (when accounts enabled).
- Secrets only environment/user-secrets; never in source control or recorded demo.
- Separate training/test data; document all synthetic generators and assumptions.
- Log correlation IDs and relevant events without private user data.
- User input displayed in HTML must be escaped to prevent script injection.
- On crash preserve persisted event log; show recoverability or explicit fail state.

Out-of-scope / scope control

প্রকল্পে বাস্তব সেনা মোতায়েন, অস্ত্রের কর্মক্ষমতা, targeting, যুদ্ধকৌশল optimization, বাস্তব সংঘাতের GIS data, কিংবা গোপন operational workflow ব্যবহার বা অনুকরণ করবে না।
Purely fictional civilian disaster-response training simulation
বলেই পরিচয় দেবে।

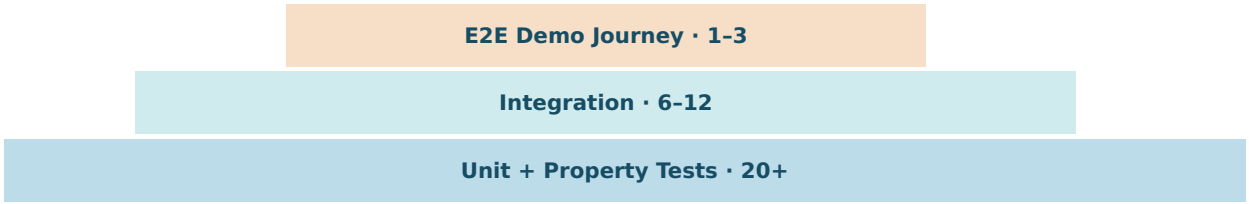
Failure matrix

Failure	Expected behavior
Duplicate command	idempotency ID prevents double effects
Invalid command while paused	typed error, state unchanged
Client disconnects	engine continues; reconnect resyncs missed events
Process crash	run marked recoverable/failed; saved events intact
Malformed scenario	validation error with field list
AI prediction unavailable	baseline estimate / “prediction unavailable”; run still works
Zero stock	warning, no negative inventory or fabricated delivery

BUILD BLUEPRINT / 16

Testing Pyramid ও Definition of Done

Test pyramid



Test ID	Scenario	Pass criteria
T01	same config + seed twice	same ordered event/state hash
T02	repeat Start / Pause command	no invalid duplicate state effect
T03	inject route block at tick 20	event persisted and observed metric recalculated
T04	no available vehicles	no illegal assignment; clear warning
T05	completed run reload	replay final hash equals original
T06	invalid JSON / numeric bounds	400 + validation details
T07	UI loses SignalR then reconnects	no missing events after backfill
T08	AI service unavailable	simulation operates without AI
T09	compare same seed + config	all metric deltas zero
T10	fresh clean machine setup	README instructions produce working demo

CI requirements

```shell
dotnet restore
dotnet build --configuration Release --no-restore
dotnet test --configuration Release --no-build
# add formatter/static analysis after baseline CI is stable
```

DoD for every feature: input validation, automated test, empty/loading/error UI state, documentation snippet, screenshot and measured success—not just “it runs on my computer”.

BUILD BLUEPRINT / 17

ল্যাপটপে Setup: ১ম দিন থেকেই হাতে-কলমে

Recommended baseline: **.NET 10 LTS + C# + ASP.NET Core + Blazor + EF Core + SQLite + xUnit**. Microsoft-এর ২০২৬-১০-১৩ support table অনুযায়ী .NET 10 LTS active; .NET 8-ও supported হলেও ২০২৬-১১-১০-এ end-of-support নির্ধারিত।

Install / verify

- Install latest stable .NET 10 SDK, Visual Studio Community (ASP.NET workload) অথবা VS Code + C# Dev Kit, Git.
- Run `dotnet --info` and `git --version`; `dotnet --list-sdks` must show a 10.x SDK.
- Create new empty GitHub repository (public only with synthetic data).

CLI foundation (run one command at a time)

Terminal / .NET 10 SDK

```
mkdir AegisSim && cd AegisSim
dotnet new sln -n AegisSim
dotnet new classlib -n AegisSim.Domain -f net10.0 -o src/AegisSim.Domain
dotnet new classlib -n AegisSim.Application -f net10.0 -o src/AegisSim.Application
dotnet new classlib -n AegisSim.Infrastructure -f net10.0 -o src/AegisSim.Infrastructure
dotnet new webapi -n AegisSim.Api -f net10.0 -o src/AegisSim.Api
dotnet new blazor -n AegisSim.Web -f net10.0 -o src/AegisSim.Web
dotnet new xunit -n AegisSim.Domain.Tests -f net10.0 -o tests/AegisSim.Domain.Tests

dotnet sln add src/*/*.csproj tests/*/*.csproj

dotnet add src/AegisSim.Application reference src/AegisSim.Domain
dotnet add src/AegisSim.Infrastructure reference src/AegisSim.Application
dotnet add src/AegisSim.Api reference src/AegisSim.Infrastructure
dotnet add tests/AegisSim.Domain.Tests reference src/AegisSim.Domain

dotnet restore && dotnet build && dotnet test
```

> Note: `dotnet sln add src/*/*.csproj` glob expansion Bash/Git Bash/WSL-এ কাজ করে; Windows PowerShell-এ প্রয়োজনে প্রতিটি .csproj আলাদাভাবে add করো। Solution template format SDK অনুযায়ী .sln বা .slnx হতে পারে; CLI auto-discovery ব্যবহার করো।

Next dependencies

Terminal

```
dotnet add src/AegisSim.Infrastructure package Microsoft.EntityFrameworkCore.Sqlite
dotnet add src/AegisSim.Infrastructure package Microsoft.EntityFrameworkCore.Design
# EF tooling, matching SDK/provider major version:
dotnet tool install --global dotnet-ef
# Blazor real-time connection: use built-in ASP.NET Core SignalR,
# register hub and group-based broadcasts in API.
```

BUILD BLUEPRINT / 18

10-Day Rapid Build Plan: প্রতিদিন কী হাতে থাকবে

Day	Concrete deliverable	Exit gate
D1	Git repo + solution + Domain models + scenario JSON	clean dotnet build; README setup
D2	Deterministic TickEngine + test fixtures	T01/T02 pass; CLI simulation visible
D3	API runs/start/pause/step/reset	Postman/curl reliable lifecycle
D4	Offline SVG map + Blazor controls	visible state transitions
D5	Event storage + SQLite migrations	run reload after restart
D6	Incident injection + history view	T03/T04 pass
D7	Snapshot/replay + event hash	T05 pass
D8	KPI dashboard + A/B comparison	repeatable 2-run delta
D9	One honest AI baseline OR better QA polish	measured ML result or 10 tests
D10	CI + Demo video + README + 3-min rehearsal	clone/run and demo without excuses

Time rule: এই ১০ দিনের পরিকল্পনা ambitious। প্রতিদিন ৪–৬ ঘণ্টা কার্যকর সময় না পেলে “D9 AI” বাদ দেবে; D1–D7 functionality আগে।

৩০ দিনের Portfolio Edition: ১০ দিনের পর

Week	Upgrade	Deliverable
Week 2	Code cleanup, better replay, multi-scenario compare	stable architecture + tests
Week 3	ML.NET prediction, grouped split, error analysis	reproducible notebook/report
Week 4	Roles, deployment, accessibility, performance	live demo / container + polished docs

BUILD BLUEPRINT / 19

Learning Path: কোন Concept আগে শিখবে?

Order	Concept	Practice exercise
1	C# basics, records, classes, interfaces, LINQ	SimState, SimEvent, IRule
2	OOP + SOLID + async/await	separate rule from repository
3	Unit tests + immutable data + seeded random	determinism and invariants
4	ASP.NET Core Web API, DI, DTO validation	create/run/commands endpoints
5	EF Core, SQLite, migrations, indexes	persist event + reload
6	Blazor components, state, SVG UI	map, dashboard, controls
7	SignalR hubs, reconnect, server push	live state updates
8	ML evaluation, baseline, leakage	predicted vs actual duration
9	CI, GitHub README, profiling	deployment evidence

Skill mastery check

- “Class vs Interface কী?”—নিজের project file দেখিয়ে ব্যাখ্যা করতে পারো।
- “Same seed-এ একই result কেন?”—Unit Test চালিয়ে proof দেখাতে পারো।
- “SignalR disconnect হলে কী হবে?”—নিজের reconnect design ব্যাখ্যা করতে পারো।
- “Database normalized না JSON কেন?”—trade-off বলতে পারো।
- “ML result কীভাবে evaluate করেছ?”—actual baseline-vs-model table দেখাতে পারো।

BUILD BLUEPRINT / 20

AI-Assisted Build: Reusable Prompt Library

AI-কে একবারে “পুরো production app বানাও” বলবে না। **একটি ছোট acceptance-tested task** দিয়ে code, test, design reasoning এবং failure modes চাইবে। প্রতিটি output নিজে compile/test করবে।

Prompt A · Architect

Copy into your AI assistant

```
You are a senior .NET software architect.
Design a small synthetic civilian emergency-response simulator.
Stack: .NET 10, C#, ASP.NET Core, Blazor, EF Core SQLite.
Priorities: deterministic fixed-tick engine, OOP, offline SVG grid,
event logging, replay and automated tests. No military operations.
Deliver: dependency diagram, domain types, interfaces, invariants,
trade-offs and acceptance tests. Do NOT generate code yet.
Assume one developer and 10-day MVP. Ask only blocking questions.
```

Prompt B · Code Generator (ONE FEATURE)

Copy into your AI assistant

```
Task: Implement a pure deterministic TickEngine in C#.
Given: current SimState, ScenarioConfig, ordered commands, seeded RNG.
Return: next SimState and ordered typed events.
Constraints: NO HttpContext, EF Core, DateTime.Now or UI references.
No negative stock; 1 tick per Step(); commands validated.
First show changed files, then COMPLETE compile-ready code.
Include xUnit tests for determinism, bounds, and edge cases.
Do not edit unrelated files; explain how to run dotnet test.
```

Prompt C · Critical Reviewer

Copy into your AI assistant

```
Review this patch as a skeptical code reviewer.
Identify correctness bugs, concurrency races, hidden state,
replay nondeterminism, SQL issues, unsafe inputs, and missing tests.
Rank severity. For each issue provide reproduction and minimal fix.
Do not praise; do not invent benchmark results.
Patch: [paste diff and current test failures]
```

Prompt D · Error Debugging

Copy into your AI assistant

```
My .NET build/test fails. Here are the FULL error logs:
[paste exact command + first real compiler error + relevant files]
Explain the root cause, smallest reliable fix, and one prevention test.
Keep the current architecture. Do not rewrite the entire project.
```

Prompt E · ML Auditor

Copy into your AI assistant

```
Audit my synthetic completion-time prediction experiment.
Inputs: feature list, group split definition, baseline MAE,
model MAE, per-scenario errors and data-generator code.
Find target leakage, data duplication and invalid performance claims.
Return a reproducible evaluation checklist; do not make up numbers.
```

AI ব্যবহার করার সঠিক loop



BUILD BLUEPRINT / 21

Interview Demo Script: ৩ মিনিটে কী দেখাবে

Time	Action on screen	কী বলবে
00:00–00:25	Architecture thumbnail + run ID / seed	“Pure C# engine and Blazor UI are separate; this is synthetic training data.”
00:25–00:55	Load Flood-01, press Start; map/events change	“State updates come from a deterministic tick engine.”
00:55–01:25	Inject closure event; watch supply/waiting KPIs	“This command is validated, persisted and visible on the timeline.”
01:25–01:50	Pause → Step → Resume; refresh browser	“The lifecycle and event log survive UI disconnection.”
01:50–02:15	Replay tick 20 and compare final checksum	“History is reproducible, not a video animation.”
02:15–02:40	Open Run A/B metrics and honest AI baseline	“AI estimates duration; observed numbers come from logged events.”
02:40–03:00	Show xUnit tests / README + 1 architecture diagram	“The demo is reproducible from a fresh clone and automated tests.”

৩০ সেকেন্ডের বাংলা পরিচিতি

> “আমি C#/.NET ব্যবহার করে একটি deterministic multi-agent emergency-response simulation platform তৈরি করেছি। এখানে scenario control, real-time updates, validated incident injection, database-backed replay, comparison report এবং automated tests আছে। AI prediction module আলাদাভাবে evaluate করেছি, তাই system AI ছাড়া চললেও engineering core সম্পূর্ণ কার্যকর।”

প্রদর্শনের সময়: ইন্টারনেটের ওপর নির্ভর না করে laptop-এ local run দেখাও। আগে থেকে database seed, sample reports এবং ২টি completed run প্রস্তুত রাখো।

BUILD BLUEPRINT / 22

Interview Q&A: তারা কী প্রশ্ন করতে পারে?

প্রশ্ন	ভালো উত্তরের মূল পয়েন্ট
কেন .NET/C#?	বিজ্ঞপ্তির সঙ্গে প্রাসঙ্গিকতা, strong typing, OOP, integrated tooling
UI-তে simulation engine লিখলে সমস্যা কী?	business rules coupling; unit test difficulty; replay inconsistency
একই seed-এ ভিন্ন result কেন হতে পারে?	unsorted iteration, wall clock, concurrency, unseeded RNG
Event sourcing vs plain log?	events must encode replayable transitions; snapshots speed up rebuilding
SQL না NoSQL কেন?	relations and consistency in MVP; JSON config/event payload compromise
Offline map কেন?	deterministic, low-dependency and reliable demonstration
SignalR কেন?	server push, groups, incremental updates, reconnect resync
LLM কি simulation decisions নেয়?	না; optional summary only; verified facts and human interpretation
Model ভালো কিনা কীভাবে জানবে?	scenario-grouped holdout, baseline MAE, error slices and limitations
আপনি AI দিয়ে code লিখেছেন?	হ্যাঁ, সহায়ক হিসেবে; আমি architecture, tests, debugging ও verification করেছি
Production করতে আর কী লাগবে?	authentication, load testing, migration plan, monitoring, accessibility

BUILD BLUEPRINT / 23

Portfolio Packaging: GitHub, CV, Demo, Evidence

README.md-এর ১২টি অপরিহার্য অংশ

- 1. ২ লাইনে problem statement ও synthetic scope
- 2. Feature matrix: Implemented / Planned আলাদা
- 3. ৩০–৪৫ সেকেন্ডের GIF বা screenshots
- 4. Architecture diagram ও dependency rules
- 5. Quickstart: exact SDK, commands, data seed
- 6. Scenario example JSON + schema validation
- 7. API table, run workflow and SignalR messages
- 8. Database ER diagram + migration instructions
- 9. Test count, commands and CI badge
- 10. Measured performance; hardware context
- 11. ML baseline/holdout metrics and limitations
- 12. License, assumptions and known issues

CV-তে লেখার নমুনা (যা সত্যি করে শেষ করবে শুধু তা-ই)

> **AEGIS-SIM | C#, .NET, Blazor, EF Core, SignalR, xUnit**
> Built a deterministic, event-driven civilian emergency-response simulator featuring scenario injection, persisted run history, replay, KPI dashboards and automated tests. Designed modular domain/application/infrastructure layers and documented reproducible test runs.

Recruitment-ready submission bundle

Artifact	Required
GitHub repository	Public (synthetic data only) + clean README
Demo video	2–4 minutes, original working UI, no fake overlays
Architecture PDF	One-page system diagram and decisions
Technical report	5–8 pages: constraints, tests, measured metrics
Running executable	Published app / clear local start script
Backup	Screen recording + screenshots + export report

> প্রকল্প শেষ হয়নি এমন feature CV বা interview-তে “implemented” বলবে না। Bug hide না করে “known limitations” দেখানোও engineering maturity-এর প্রমাণ।

BUILD BLUEPRINT / 24

Risk Register: কোথায় আটকে যেতে পারো

Risk	Likelihood	Mitigation / fallback
C# unfamiliarity	High	first 2 days core OOP + mini-engine; AI features defer
Blazor UI takes too long	Medium	minimal Razor screen + SVG; skip external UI library
SignalR broken during demo	Medium	polling fallback 1–2 sec / run locally
Replay mismatch	High	immutable events, ordering, golden tests from day 2
DB migration conflicts	Medium	clean migrations, backup seed and local recreate script
ML data leakage	High	hold out whole scenario families; keep baseline always
Too many features	High	MVP acceptance criteria first; fixed scope freeze after D7
Performance poor	Medium	reduce entity count; profile, then throttle UI updates
Internet or model API unavailable	High	offline map and deterministic template report
Claims too ambitious	Medium	mark demo-only, synthetic, experimental features plainly

টাকা ও Hardware

Visual Studio Community/VS Code, .NET SDK, Git, SQLite, xUnit, Blazor এবং local SVG map দিয়েই মূল প্রজেক্ট **শূন্য সফটওয়্যার লাইসেন্স খরচে** করা সম্ভব (নিজের কম্পিউটার/বিদ্যুৎ/ইন্টারনেট বাদে)। Optional LLM API বা cloud deployment দরকার নেই। ১৬ GB RAM-র সাধারণ ডেভেলপমেন্ট ল্যাপটপে ছোট MVP লক্ষ্য রাখো; কার্যক্ষমতা বাস্তবে পরীক্ষা করবে।

BUILD BLUEPRINT / 25

Final Acceptance Gate: “World-Class” Proof Checklist

- ☐ Clean clone → build → run → demo instructions work.
- ☐ Exactly one authoritative SimulationState per run.
- ☐ 2 reproducible runs share the same seed and result hash.
- ☐ Invalid command cannot corrupt state.
- ☐ Incident injection changes observed metrics and creates event.
- ☐ Event history persists after restart; replay verifies checksum.
- ☐ Map, event feed and KPI show consistent tick number.
- ☐ At least 10 useful tests run successfully in CI.
- ☐ Report numbers recomputed from logs; not hand-edited.
- ☐ ML has actual baseline metric—or is clearly marked planned.
- ☐ No real-world military, private, secret or sensitive dataset.
- ☐ A 3-minute live demo works without internet.
- ☐ GitHub README contains screenshots, architecture and limitations.
- ☐ You can explain every core class and one difficult bug you fixed.

আগামী ২ ঘণ্টায় যে ৫টি কাজ করবে

- Create GitHub repo এবং .NET solution skeleton.
- MiniEngine C# code নিজের IDE-তে compile করে Console বা Test দিয়ে চালাও.
- Flood-01 JSON scenario লিখো: ৩ shelters, ৮ synthetic resources, ২ scheduled incidents.
- Same-seed deterministic unit test যোগ করো.
- README-তে “Implemented: engine skeleton / Planned: UI, AI” স্পষ্টভাবে লেখো.

WINNING PRINCIPLE

Small, correct, reproducible, demonstrable software > giant, unfinished “AI-powered” concept.

APPENDIX / 26

Official Learning References / Sources

লিংকগুলো Microsoft-এর অফিসিয়াল ডকুমেন্টেশন বা মূল platform reference। প্রযুক্তির API পরিবর্তিত হতে পারে; নিজের SDK version-এর সাথে মিলিয়ে ব্যবহার করো।

01 · .NET supported versions (2026-10-13)
https://dotnet.microsoft.com/en-us/platform/support/policy

02 · .NET getting started
https://learn.microsoft.com/en-us/dotnet/core/get-started

03 · C# language guide
https://learn.microsoft.com/en-us/dotnet/csharp/

04 · ASP.NET Core fundamentals
https://learn.microsoft.com/en-us/aspnet/core/fundamentals/

05 · SignalR overview
https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction

06 · SignalR groups and users
https://learn.microsoft.com/en-us/aspnet/core/signalr/groups

07 · Blazor fundamentals
https://learn.microsoft.com/en-us/aspnet/core/blazor/fundamentals/

08 · EF Core SQLite provider limitations
https://learn.microsoft.com/en-us/ef/core/providers/sqlite/limitations

09 · ML.NET documentation
https://learn.microsoft.com/en-us/dotnet/machine-learning/

10 · ASP.NET Core integration tests
https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests

11 · .NET CLI
https://learn.microsoft.com/en-us/dotnet/core/tools/

12 · OWASP Top 10
https://owasp.org/www-project-top-ten/

এই নথি বিশেষভাবে একটি
portfolio educational project
-এর জন্য। নিয়োগ বিজ্ঞপ্তির প্রকৃত শর্ত, প্রজেক্টের কাজের পরিধি এবং কর্তৃপক্ষের অভ্যন্তরীণ প্রয়োজনের মধ্যে পার্থক্য আছে। নিয়োগের নিশ্চয়তা নেই।